Module 9: Tech 101 and System Design for PMs

Detailed supplement: how a digital product works behind the screen, requirements, trade-offs, failures, and a system design assignment



Written by Varun Kumar, for SkillNavora · Draft

Official site: www.skillnavora.in

Prepared for Skillnavora — Learn. Practise. Implement. Grow.

Coverage map: what is new and what is already in the module

The main Module 9 handbook covers system design, APIs, databases and latency in depth. This supplement adds the deck's teaching sequence: one login flow traced through every layer, the four levels of tech understanding, requirements and trade-offs, failure design, and the system design assignment.

Topic (Module 9 deck)	Handbook section	Status here
Four levels of tech understanding	—	New: what each level means for a PM
One user action traced through seven steps	—	New: the request journey, with a login example
Frontend, API, backend, database, third party	9.x (partial)	Extended: what the PM should ask at each layer
Status codes and user experience	9.x (partial)	New: each code with the right user response
SQL versus NoSQL for product decisions	9.x	Extended: four data types and their use
Caching, CDN, async, logging, monitoring	9.x	Extended: the PM's view of each
Functional versus non-functional requirements	—	New: weak versus strong requirements
Product trade-offs and build versus buy	—	New: six trade-offs, and five build-or-buy questions
Monolith versus microservices	9.x (partial)	New: when each fits, with a worked example
Backlog grooming, estimation, sprint planning	—	New: how a PM prepares work for the team
Failure design and AI feature trade-offs	—	New: fallback design and AI summary requirements
Security, privacy, reliability and cost	—	New: checkpoints and the hidden cost of tech choices
System design assignment (six steps)	—	New: the brief, step by step

NOTE ON THE SOURCE

Several deck slides are pictures with no text, so the supplement uses the diagrams' labels and the speaker prompts. Examples are teaching cases, not company data.

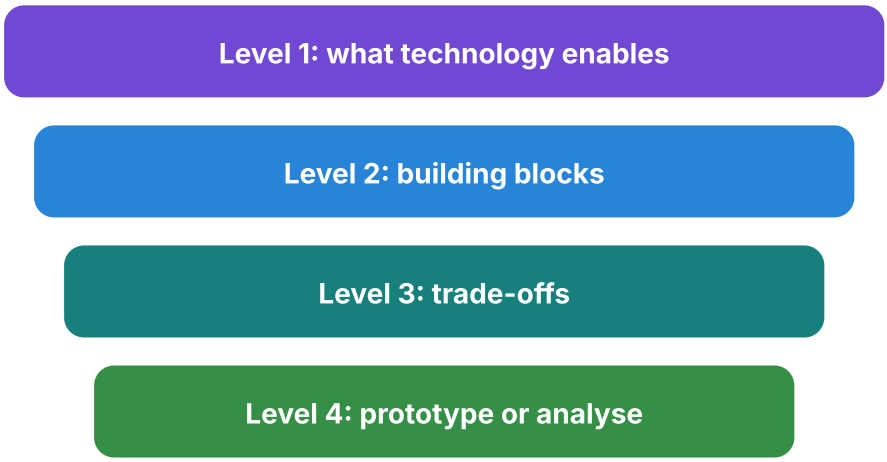
NEW · FOUNDATIONS

Four levels of tech understanding

Not every PM needs to code. Every PM should know enough tech to make better product decisions. The deck names four levels:

Level	What it means	What it lets you do
1. Understand what technology enables	Know which problems technology can solve	Spot product opportunities
2. Understand the building blocks	Know logic, data, and APIs by name	Discuss feasibility with engineers
3. Understand trade-offs and design choices	Weigh options and their costs	Make stronger scope decisions
4. Build prototypes or analyse data	Make a working prototype or query the data yourself	Validate ideas faster

Aim: most PMs should reach level 2 and build level 3 over time.



The four levels of tech understanding

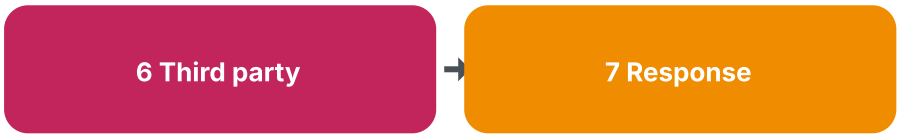
NEW · ONE ACTION, SEVEN STEPS

Every product action creates a system journey

The deck traces a single action, a user tapping a button, through the system. The user sees only the first and last steps.



From a tap to a stored result



The rest of the journey

Worked example: OTP login

Entering a phone number and tapping Send OTP touches each layer:

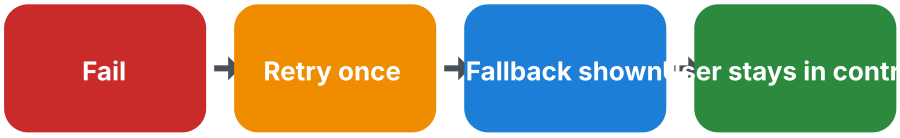
Layer	What it does in the login	Question a PM should ask
Frontend	Collects the number, validates the format, shows loading and errors	What should the button say? When should resend appear?
API	Sends a structured request (phone number, device ID) and expects a structured response	What happens when the request fails or is slow?
Backend	Validates, generates the code, applies rate limits, creates a session, logs the event	What must be logged for security?
Database	Stores the user, the attempt, the verification state and the session	What data must be kept, and for how long?
Third party	An SMS provider delivers the code and reports status	What if the message is delayed or fails?

NEW · FAILURES AND STATUS CODES

API errors are user experience problems

Status code	Meaning	What the user should see
200	Success	The result
400	Bad request	A clear message to fix the input
401	Unauthorised	A prompt to log in
404	Not found	An unavailable state with a way back
500	Server error	A retry or a fallback
503	Service unavailable	A temporary notice with a time estimate

A PM does not only define the happy path. A PM also defines how failure should feel. Ask: what should the error say, should the system retry on its own, and what fallback should the user see?



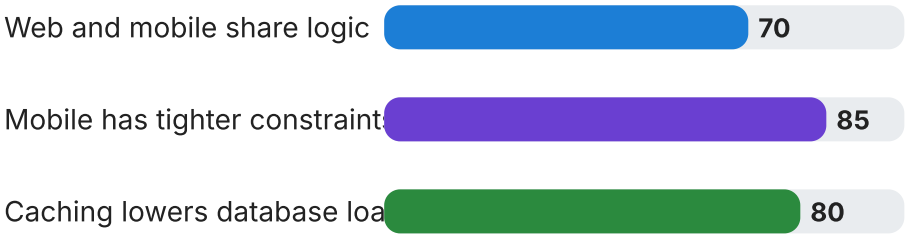
A failure path, designed in advance

EXTENDED · DATA AND ARCHITECTURE

Databases, SQL and NoSQL, and how the system is built

A database stores the product state that makes each experience possible. The deck's typical questions: what data gets saved, what gets updated, and what must be recoverable.

Store type	Best for	PM implication
SQL (tables)	Orders, payments, CRM	Clear reporting; changes need planning
Document (flexible records)	Profiles, content, settings	Adapts quickly to change
Graph (relationships)	Social networks, recommendations	Relationship-heavy features
Key-value (fast lookups)	Sessions, OTPs, cache	Very fast; simple queries



Illustrative emphasis, not measured data

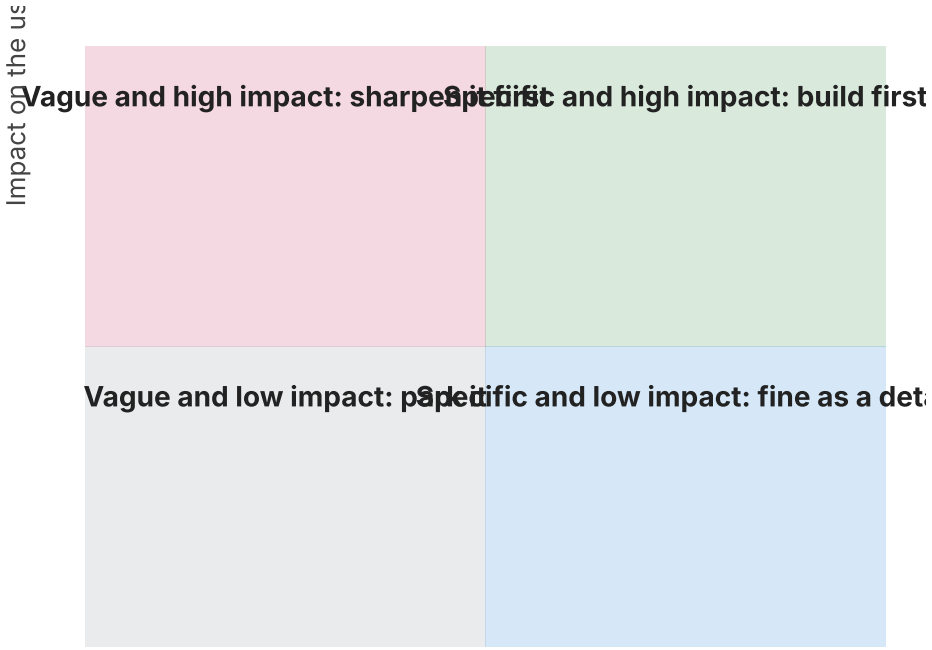
Monolith versus microservices: a monolith keeps the product in one deployable app, which is simple early on. Microservices split each capability into its own service, which helps when several teams and high scale arrive, at the cost of more coordination and monitoring.

NEW · REQUIREMENTS

Functional versus non-functional requirements

A functional requirement says what the system should do. A non-functional requirement says how well it should work. A feature is not complete until both are defined.

Weak requirement	Stronger requirement
'The app should be fast'	Search results load within 2 seconds for 95 percent of searches
'User can upload a file'	User can upload PDF files up to 25 MB, with progress, retry and an error state
'User gets a notification'	User receives a payment confirmation within 10 seconds of a successful payment
'The system should be secure'	Sensitive payment data is not stored unless compliance requires it



Sharpen the requirements that matter most

Write each non-functional requirement with a number, a condition and a time window, so QA can test it.

NEW · TRADE-OFFS

Trade-offs and build versus buy

Trade-off	The question
Speed versus accuracy	Should the AI answer faster or more correctly?
Build versus buy	Should we build this or use a vendor?
Real-time versus delayed	Does the user really need instant updates?
Cost versus performance	Should we spend more infrastructure to improve speed?
Security versus convenience	Should login be stricter or smoother?
MVP versus scale-ready	Should we build fast, or future-proof?

Build versus buy: five decision questions



Five questions before you decide

PM RULE OF THUMB (FROM THE DECK)

Build what makes you different. Buy what helps you move faster.

NEW · TEAM FLOW

Backlog grooming, estimation and sprint planning

Grooming turns rough ideas into work the team can build. The deck's steps: clarify the user problem, rewrite it as a user story, add acceptance criteria, and split large work into smaller stories.



Backlog grooming in four steps

Estimation uses relative points: 1 to 2 points for small, clear work; 3 to 5 for medium work with some unknowns; 8 or more for large, uncertain work, which should usually be split. **Sprint planning** picks the goal, reviews the top items, checks effort and capacity, and commits to a sprint plan the team can finish.

NEW · AI FEATURES

AI summary: trade-offs and requirements

An AI feature forces product choices as well as model choices. The deck's meeting-summary example contrasts real-time with post-meeting summaries, a third-party model with an in-house one, keeping the transcript with deleting it, auto-sending with review first, and detailed with short summaries.

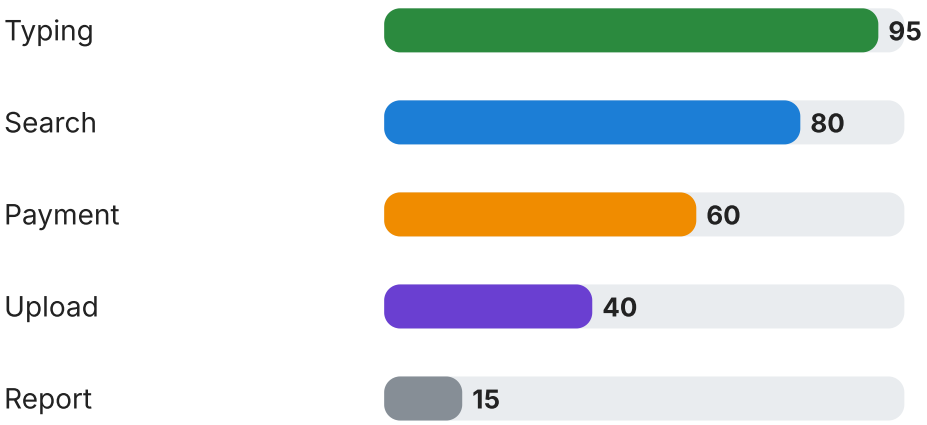
Requirement	What to define
Functional	Generate a summary and extract action items
Latency	Summary ready within 2 minutes after the meeting ends
Accuracy	Users can review and edit before sharing
Privacy	No sharing or storage of the transcript without consent
Fallback	If AI fails, show the transcript and mark the summary unavailable
Evaluation	Track edit rate, share rate and a helpfulness rating
Risk	Do not auto-assign action items without user review

NEW · LATENCY AND EXPECTATIONS

Latency is expectation design

Latency is not only speed. It is how long the user can wait before the experience starts to break. Typing feels instant; search results feel fine within one to two seconds; a monthly report can be slow, as long as the user knows it is coming.

Action	Acceptable wait	What to show while waiting
Typing a message	Instant	Nothing extra
Search results	1 to 2 seconds	A quiet loading state
Payment confirmation	A few seconds	Progress and a clear state
Video upload	Longer, with progress	A progress bar and a cancel option
AI image generation	Longer, with a queue	Queue position and status
Monthly report	Delay is fine	A note that it is being prepared



How much delay each action can tolerate (illustrative)

NEW · RELIABILITY AND FALLBACKS

Design the fallback, not only the happy path

When this fails	Offer this instead
Payment gateway down	An alternate payment method
SMS OTP delayed	Resend and a voice option
Delivery tracking unavailable	The last known status
AI summary failed	The transcript only
Search service down	Cached or recommended results
Vendor API failed	A graceful fallback state

Security and privacy are product decisions too. The deck's checkpoints: authentication (who is this user?), authorisation (what can they access?), consent, data minimisation, encryption, abuse prevention and compliance.

Cost: every technical decision has a cost: development, infrastructure, operations, maintenance and the opportunity cost of what you did not build.

ASSIGNMENT

System design brief: an online education platform

You are a product manager at an online education platform. Work through six steps: define the user stories, list functional and non-functional requirements for each area, outline the core system design (the main components that deliver the user stories), identify risks, and finish with assumptions and future plans.

1. Write the main user stories for learners and teachers.
2. List one or two non-functional requirements in each category (speed, availability, security, privacy).
3. Outline the core components: frontend, API, backend, database, third-party services.
4. Trace one learner action (for example, joining a live class) through each layer.
5. Name the three biggest risks, and one fallback for each.
6. List your assumptions and the improvements for version 2.

SELF-CHECK

Does each requirement have a number? Did you trace one action through every layer? Does each risk have a fallback?

Quiz with answers

1. Name the seven steps a user action passes through.
   Answer: user action, frontend, API, backend, database, third party, response.
2. What does a 401 status mean for the user?
   Answer: the user is not logged in; show a prompt to log in.
3. How is a non-functional requirement different from a functional one?
   Answer: a functional requirement says what the system does; a non-functional one says how well it does it.
4. When is a monolith a good choice?
   Answer: an early product where speed and simplicity matter more than independent services.
5. What is the PM rule of thumb for build versus buy?
   Answer: build what makes you different; buy what helps you move faster.

MODULE 9 · FINAL

Completion checklist

Completion checklist

Done	Item
☐	Read the coverage map
☐	Traced one product action through every layer
☐	Wrote two requirements, functional and non-functional, with numbers
☐	Made one build-or-buy decision with the five questions
☐	Designed a fallback for one failure
☐	Completed the system design brief